WEB DEVELOPMENT · 3RD SEMESTER

Week 01 · Block 1

How the Web Works

<web-development> · Hochschule Ansbach
Matthias Kellnhofer

After this block, you can…

- Explain the **client/server** model and the **request/response** cycle
- Read a **URL** and name each of its parts
- Describe an **HTTP request** and **response** — methods, status codes, headers
- Open the browser **dev tools** and watch real traffic in the Network tab

No code in this block. First, we build the mental model everything else in this course sits on.

You enter a URL in your Browser,
and the page appears.

But what happens **in between**?

By the end of the hour, you'll watch every step of it yourself.

PART 1

Client & Server

Two roles

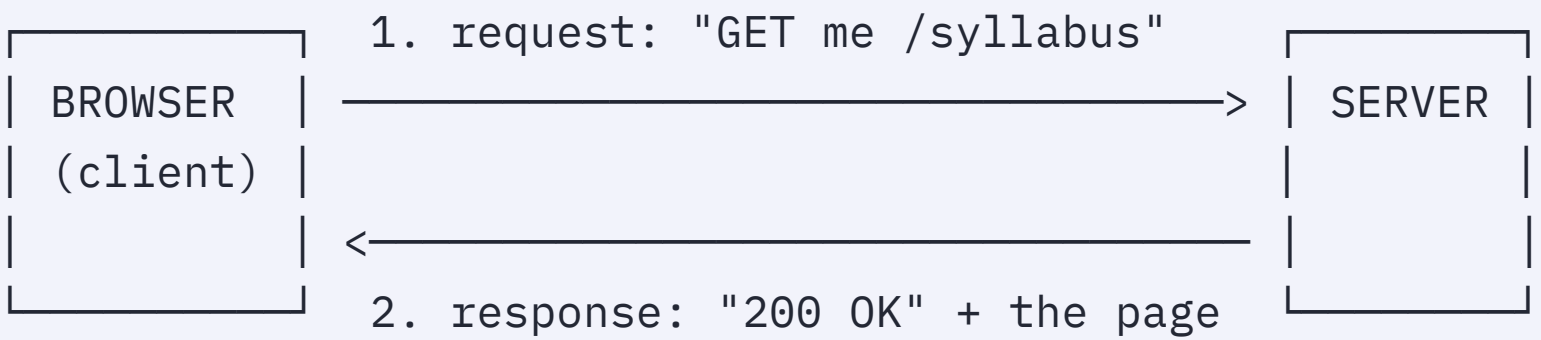
The web is a conversation between two parties:

- **Client** — asks for things. Your **browser** is a client.
- **Server** — a program on some machine that **answers** requests.

The client always starts the conversation. The server only ever **responds**.

> "Server" is a *role*, not a big machine in a basement. In exercise 0, **your laptop** already was one. In Block 7, you write your own server.

The request/response cycle



One request → one response. That's the whole loop.

A single web page usually means **many** of these loops — one per file.

Stateless by default

Each request/response is **independent**. The server does not remember the last one.

- Request for the page, request for the stylesheet, request for the logo — the server treats each as a stranger.
- Nothing is "logged in" at the HTTP level.

> There's no object holding state between calls. Keeping a user "logged in" is something we have to *build* — that's **<sessions>**, in Block 7b.

PART 2

The URL

Uniform Resource Locator

Anatomy of a URL

A URL is the **address** of one resource. Every part answers a question:

```
https://wd.hs-ansbach.de:443/syllabus?week=1#intro
```

- `https` — **scheme**: which protocol to speak
- `wd.hs-ansbach.de` — **host**: which server to talk to
- `:443` — **port**: which door (443 = HTTPS default, usually omitted)
- `/syllabus` — **path**: which resource on that server
- `?week=1` — **query**: extra parameters
- `#intro` — **fragment**: a spot on the page — **never sent to the server**

From name to machine

`wd.hs-ansbach.de` is for humans. The network needs a number — an **IP address**.

- **DNS** (Domain Name System) is the web's phone book: name → IP.
- The browser asks DNS *before* it can send the request.

We won't go deeper than this — just know the name-to-number lookup happens first.

PART 3

HTTP

Hypertext Transfer Protocol

HTTP is just text

HTTP is the language client and server speak. A message is plain, readable text.

A **request** has three parts:

1. A **request line** — method + path + version
2. **Headers** — metadata, one per line
3. An optional **body** — data being sent

Let's look at a real one.

A request, up close

```
GET /syllabus HTTP/1.1
Host: wd.hs-ansbach.de
User-Agent: Mozilla/5.0 (Firefox)
Accept: text/html
```

- **`GET`** — the method (what to do)
- **`/syllabus`** — the path (which resource)
- **`Host`**, **`Accept`**, … — headers describing the request
- No body here — a `GET` just asks

The response, up close

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
Content-Length: 1024

<!DOCTYPE html>
<html> … the page … </html>
```

- `200 OK` — the **status line**: how it went
- **Headers** — describe the body (`Content-Type` tells the browser it's HTML)
- A blank line, then the **body** — the actual content

Headers, briefly

Headers are **key: value** metadata on both requests and responses.

On requests

- `Host` — which site
- `Accept` — formats I understand
- `Cookie` — data from before

On responses

- `Content-Type` — what this body is
- `Content-Length` — how big
- `Set-Cookie` — remember this

You rarely write these by hand — the browser and server fill most in. But you'll **read** them constantly when debugging.

PART 4

HTTP Methods

What do you want to do?

The **method** is the verb of the request:

- **`GET`** — *fetch* a resource. No side effects. (Typing a URL, clicking a link.)
- **`POST`** — *send* data to create or submit. (Submitting a form.)
- **`PUT`** / **`PATCH`** — *update* an existing resource.
- **`DELETE`** — *remove* a resource.

Rule of thumb: `GET` only *reads* — it should never change anything on the server. The other methods (`POST`, `PUT`, `PATCH`, `DELETE`) are the ones that *write*. We'll lean on `GET` and `POST` from Block 7 on.

GET vs POST

GET

- Data rides in the **URL**: `?week=1`
- Visible, bookmarkable, shareable
- For **reading**

POST

- Data rides in the **body**
- Not in the URL
- For **writing** / larger or sensitive data

Neither is "secure" on its own — hiding data in the body is *not* encryption. That's what HTTPS is for. More in Block 7.

PART 5

HTTP Status Codes

Five families

The first digit tells you the category:

- **1xx** — informational *(rare)*
- **2xx** — **success**
- **3xx** — **redirection** — "look elsewhere"
- **4xx** — **client error** — *you* sent something wrong
- **5xx** — **server error** — the *server* broke

4xx vs 5xx: 4xx = "you messed up", 5xx = "the server messed up". Knowing which side to blame saves hours of debugging.

The ones you'll actually meet

not examined

- `200 OK` — here's what you asked for
- `301` / `302` — moved / redirected (browser follows automatically)
- `304 Not Modified` — "you already have it" → served from cache
- `400 Bad Request` — malformed request
- `403 Forbidden` — you're not allowed
- `404 Not Found` — no such resource *(the famous one)*
- `500 Internal Server Error` — the server threw an exception

PART 6

See It Yourself

The browser dev tools

Everything so far is happening on every page you visit. Let's watch it.

- Open dev tools: `F12`, or right-click → **Inspect**
- Go to the **Network** tab
- **Reload** the page — every request shows up as a row

The dev tools are the single most useful instrument in this course. We'll live in them.

Reading the Network tab

Each row is one request/response. The key columns:

- **Name** — the file requested
- **Status** — the response code (200, 304, 404…)
- **Method** — GET, POST, …
- **Type** — document, stylesheet, image, font…
- **Size** — bytes transferred *(or "disk cache")*
- **Time** — how long it took

Click any row → **Headers**, **Response**, **Timing** — the raw message, exactly as we drew it.

One page, many requests

The **first** row is the HTML **document**. Everything after it is a resource the HTML asked for:

```
index.html   ← the document (1 request)
  ├─ styles.css      ← a stylesheet
  ├─ logo.svg        ← an image
  └─ favicon.svg     ← the tab icon
```

The browser reads the HTML, discovers these links, and fires off **more** request/response loops — exactly the cycle from Part 1, many times over.

Recap

- The web is **client ⇄ server**, one **request/response** at a time — and **stateless**
- A **URL** says *where*; the **method** says *what*; the **status code** says *how it went*
- **HTTP messages** are just text: request line / status line, headers, body
- The **Network tab** lets you watch all of it live

<EXERCISE> · 45 MIN

Your turn 👉

Become a traffic inspector: watch real HTTP — on a real website, and on your own server.

1. Watch a real page load in the **Network tab**
2. Read one request in detail — method, status, headers
3. Trigger a **404** (and a redirect) on purpose
4. **Be the server:** serve a small page, watch its requests, and find the **304**

At home — Task 5: map a request & response **by hand**.

Starter: `exercises/week01/1/` — open `README.md` and follow along.